# Recitation 10 – Routing Protocols

Daniel Karalnik and Liam Tal

Computer Networks

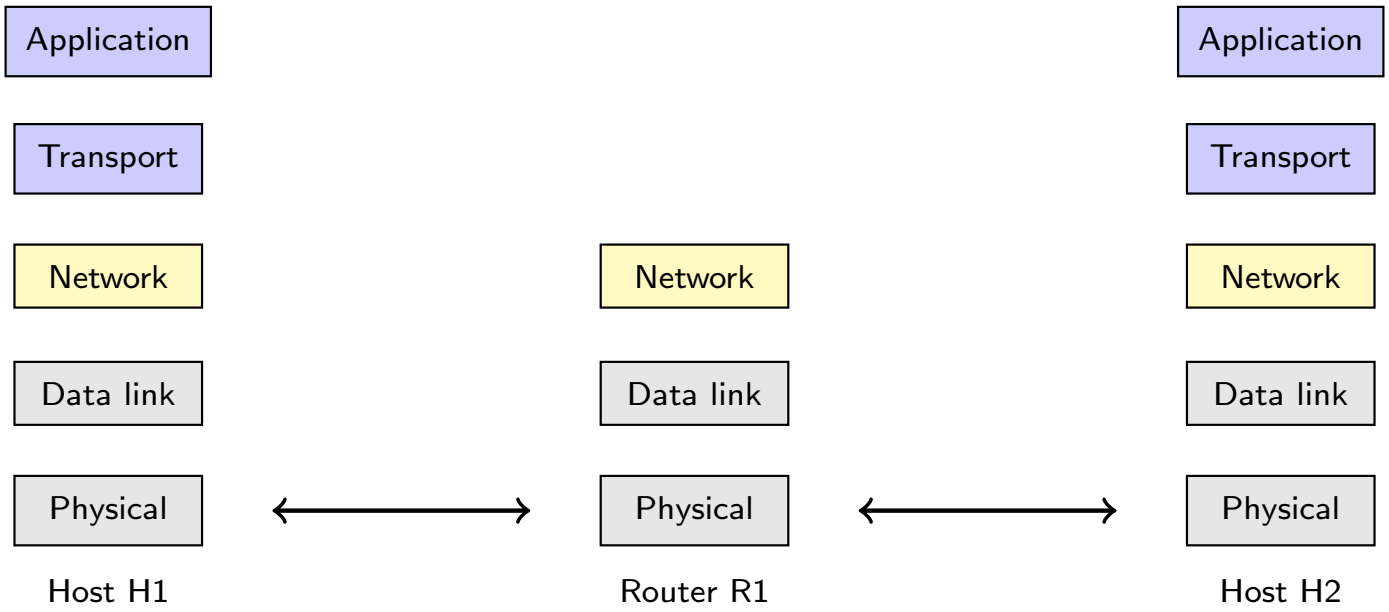
# Table of Contents


1. Network Layer Recap
2. Link-State (LS) Algorithm
3. Distance-Vector (DV) Algorithm
4. LS vs DV Comparison
5. Practice Questions
6. True/False Questions

# Today

- Network Layer Recap
- Link-State (LS) Algorithm
- Distance-Vector (DV) Algorithm
- LS vs DV Comparison
- Practice Questions
- True/False Questions

# Network Layer Recap



# Network Layer Recap

**Transport vs Network Layer:**

- **Transport Layer**: Process-to-process communication
- **Network Layer**: Host-to-host communication

**Routing:**

> Determine the route or path taken by packets as they flow from a sender to a receiver.

**Routing Algorithms:**

- **Link-State (LS)**
- **Distance-Vector (DV)**

# Link-State (LS) Algorithm

## Link-State Routing

A **global** routing algorithm where each router has **complete knowledge** of the network topology and link costs.

**Characteristics:**

- Each node broadcasts link-state packets to **all** other nodes
- All nodes have **identical** information about the network
- Uses **Dijkstra's algorithm** to compute shortest paths

**Complexity:**

- **Runtime**: $O(n^2)$ naive, $O(n \log n)$ with heap
- **Messages**: $O(nE)$ initially, update sent to **all nodes**

# Dijkstra's Algorithm

## Algorithm Overview

Computes the least-cost path from one node (source) to all other nodes in the network.

**Notation:**

- $D(v)$: Current cost of path from source to $v$
- $p(v)$: Predecessor node along path from source to $v$
- $N'$: Set of nodes whose least-cost path is known

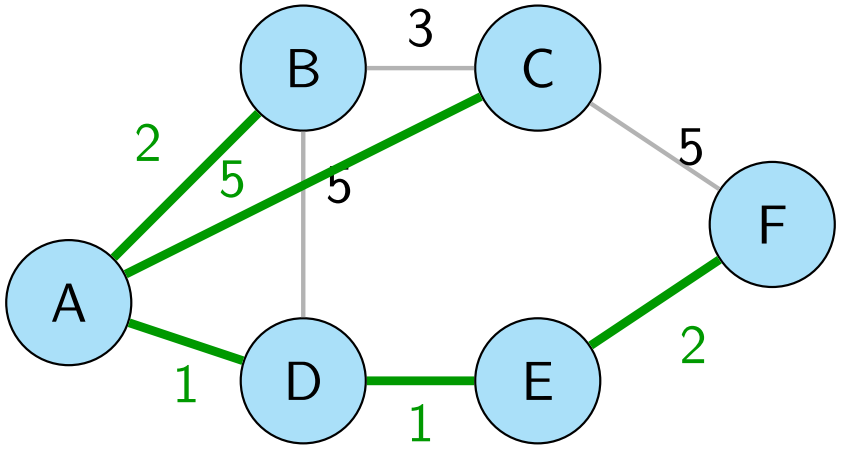
# Dijkstra's Algorithm - Pseudocode

```
function Dijkstra(Graph, source):
  Initialize:
    N' = {source}
    for all nodes v:
      if v is neighbor of source:
        D(v) = c(source, v)
        p(v) = source
      else:
        D(v) = infinity

  Loop until all nodes in N':
    w = node not in N' with minimum D(w)
    add w to N'
    for all neighbors v of w not in N':
      if D(w) + c(w,v) < D(v):
        D(v) = D(w) + c(w,v)
        p(v) = w

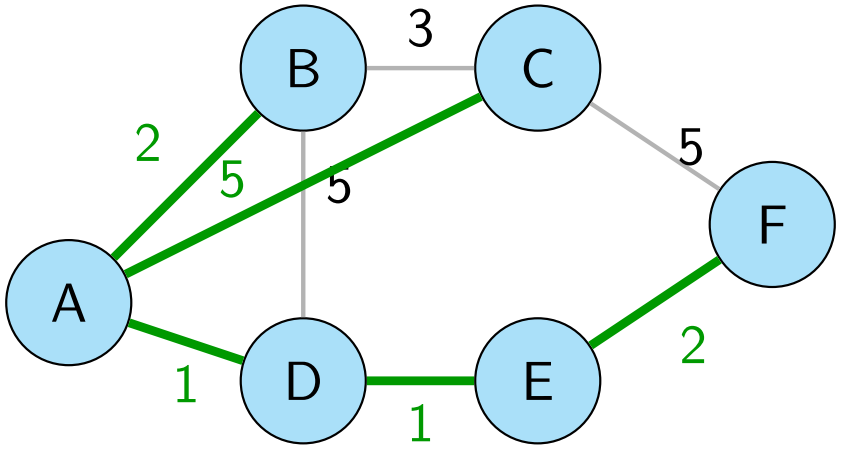
  return D, p  // distances and predecessors
```

# Link-State Example



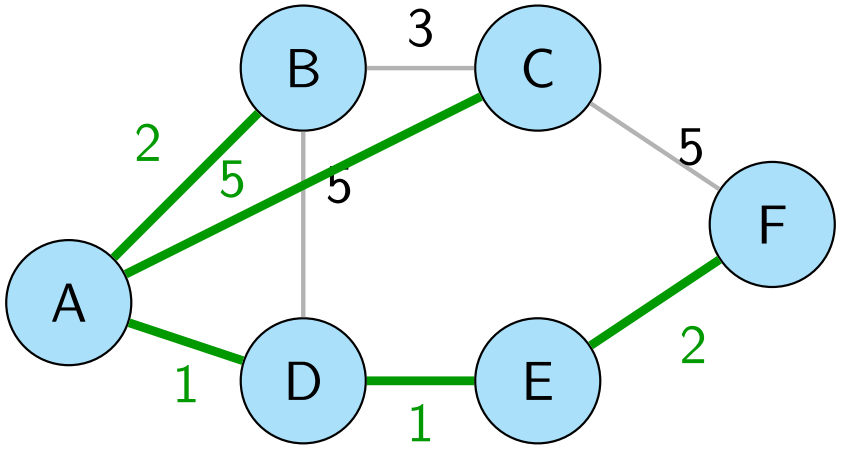

| Step | N' | D(B),p(B) | D(C),p(C) | D(D),p(D) | D(E),p(E) | D(F),p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2,A | 5,A | 1,A | $\infty$ | $\infty$ |
| | | | | | | |

# Link-State Example



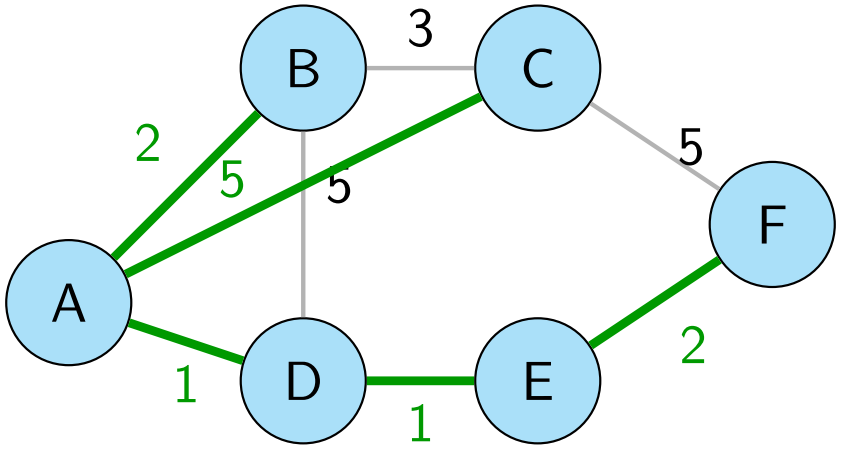

| Step | N' | D(B),p(B) | D(C),p(C) | D(D),p(D) | D(E),p(E) | D(F),p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2,A | 5,A | 1,A | $\infty$ | $\infty$ |
| 1 | AD | 2,A | 5,A | | 2,D | $\infty$ |
| | | | | | | |

# Link-State Example



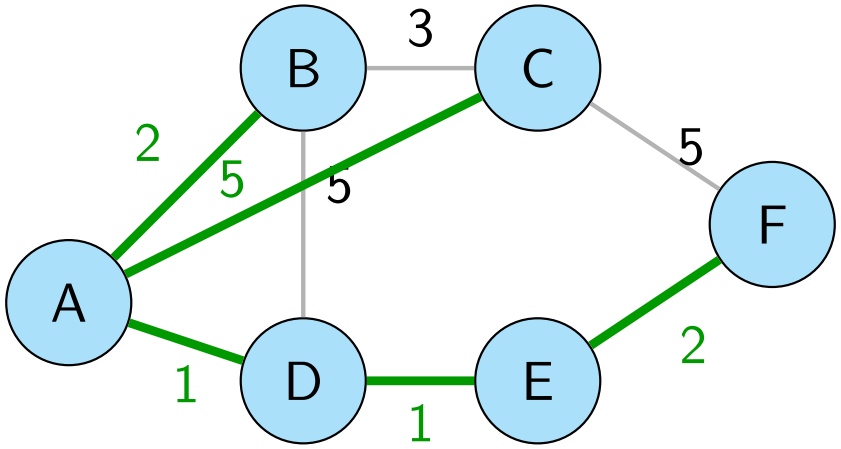

| Step | N' | D(B),p(B) | D(C),p(C) | D(D),p(D) | D(E),p(E) | D(F),p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2,A | 5,A | 1,A | $\infty$ | $\infty$ |
| 1 | AD | 2,A | 5,A | | 2,D | $\infty$ |
| 2 | ADE | 2,A | 5,A | | | 4,E |
| | | | | | | |

# Link-State Example



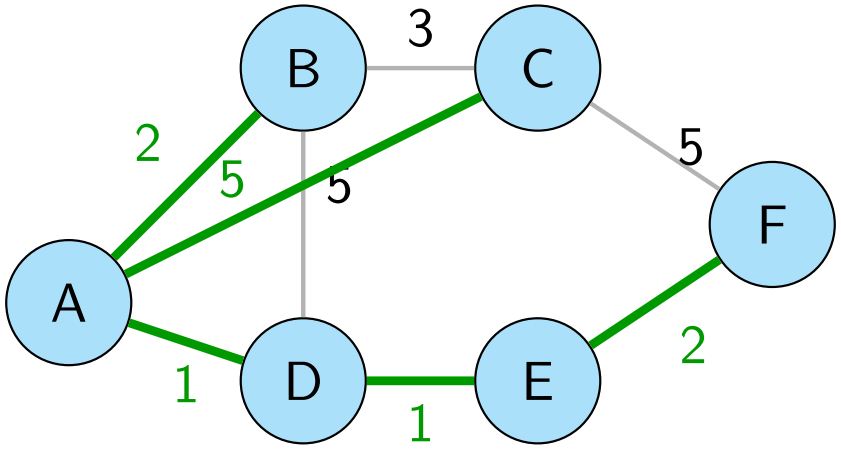

| Step | N' | D(B),p(B) | D(C),p(C) | D(D),p(D) | D(E),p(E) | D(F),p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2,A | 5,A | 1,A | $\infty$ | $\infty$ |
| 1 | AD | 2,A | 5,A | | 2,D | $\infty$ |
| 2 | ADE | 2,A | 5,A | | | 4,E |
| 3 | ADEB | | 5,A | | | 4,E |
| | | | | | | |

# Link-State Example




| Step | N' | D(B),p(B) | D(C),p(C) | D(D),p(D) | D(E),p(E) | D(F),p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2,A | 5,A | 1,A | $\infty$ | $\infty$ |
| 1 | AD | 2,A | 5,A | | 2,D | $\infty$ |
| 2 | ADE | 2,A | 5,A | | | 4,E |
| 3 | ADEB | | 5,A | | | 4,E |
| 4 | ADEBF | | 5,A | | | |
| | | | | | | |

# Link-State Example




| Step | N' | D(B),p(B) | D(C),p(C) | D(D),p(D) | D(E),p(E) | D(F),p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2,A | 5,A | 1,A | $\infty$ | $\infty$ |
| 1 | AD | 2,A | 5,A | | 2,D | $\infty$ |
| 2 | ADE | 2,A | 5,A | | | 4,E |
| 3 | ADEB | | 5,A | | | 4,E |
| 4 | ADEBF | | 5,A | | | |
| 5 | ADEBFC | | | | | |

**Green edges** = Shortest Path Tree from A

# Distance-Vector (DV) Algorithm

## Distance-Vector Routing

A **distributed**, **asynchronous**, and **iterative** routing algorithm where each router only knows its neighbors.

**Characteristics:**

- **Distributed**: Each node computes its own routing table
- **Asynchronous**: Nodes don't need to operate in lockstep
- **Iterative**: Process continues until no more updates

## Bellman-Ford Equation

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

# Distance-Vector Algorithm

**Each node $x$:**

1. Maintains distance vector $D_x = [D_x(y) : y \in N]$
2. Maintains neighbors' distance vectors
3. Periodically sends its DV to neighbors

**When node $x$ receives a DV from neighbor $v$:**

1. Save $v$'s distance vector
2. Recalculate own DV using Bellman-Ford:

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\} \text{ for all } y \in N$$

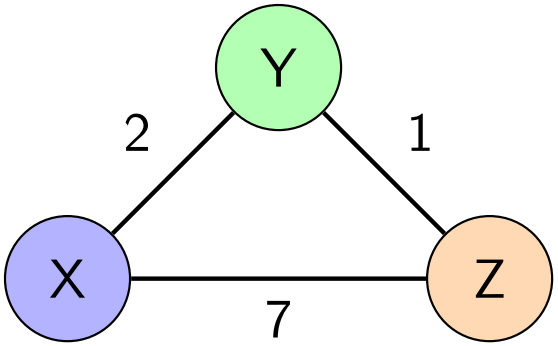
3. If DV changed, notify neighbors

**Key Insight**: Each node only needs to know:

- Cost to each neighbor
- Neighbor's distance vector

# Distance-Vector: Message Propagation

**Network Topology:**




**Neighbors:**

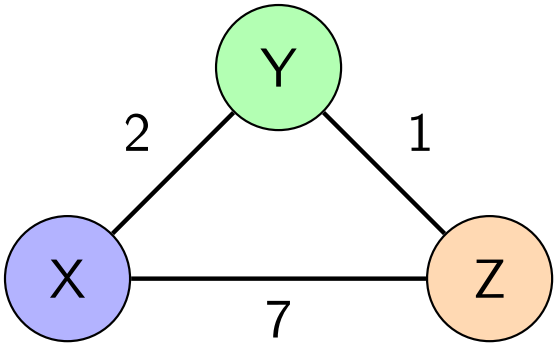
- X: Y, Z
- Y: X, Z
- Z: X, Y

**Message Exchange Over Time:**

$t_0$: Each node knows only direct link costs

# Distance-Vector: Message Propagation

## Network Topology:




## Neighbors:

- X: Y, Z
- Y: X, Z
- Z: X, Y

## Message Exchange Over Time:

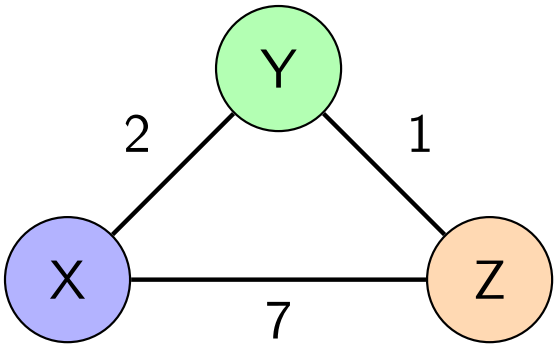
$t_0$: Each node knows only direct link costs

$t_1$: Nodes exchange DVs with neighbors

- X sends $D_X$ to Y and Z
- Y sends $D_Y$ to X and Z
- Z sends $D_Z$ to X and Y

# Distance-Vector: Message Propagation

## Network Topology:




## Neighbors:

- X: Y, Z
- Y: X, Z
- Z: X, Y

## Message Exchange Over Time:

$t_0$: Each node knows only direct link costs

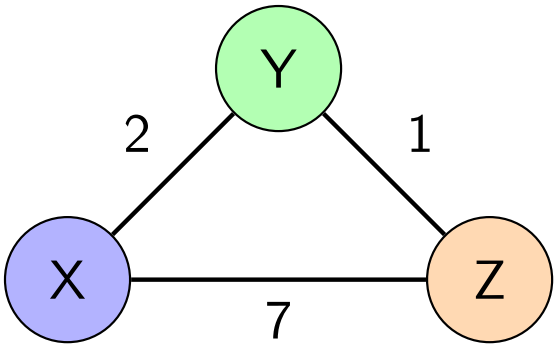
$t_1$: Nodes exchange DVs with neighbors

- X sends $D_X$ to Y and Z
- Y sends $D_Y$ to X and Z
- Z sends $D_Z$ to X and Y

$t_2$: Nodes update DVs using Bellman-Ford

# Distance-Vector: Message Propagation

## Network Topology:




## Neighbors:

- X: Y, Z
- Y: X, Z
- Z: X, Y

## Message Exchange Over Time:

$t_0$: Each node knows only direct link costs

$t_1$: Nodes exchange DVs with neighbors

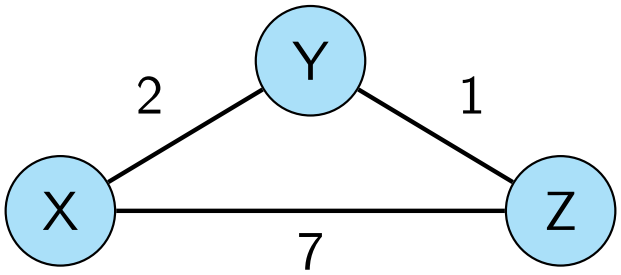
- X sends $D_X$ to Y and Z
- Y sends $D_Y$ to X and Z
- Z sends $D_Z$ to X and Y

$t_2$: Nodes update DVs using Bellman-Ford

$t_n$: Repeat until no changes (converged)

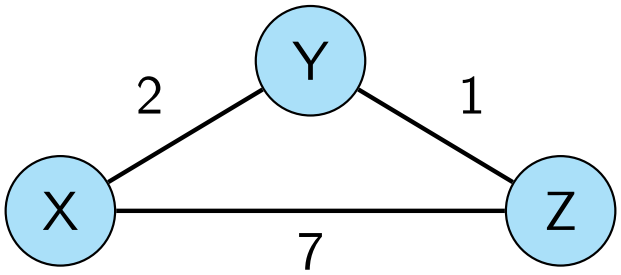
**Key**: Each node only sends to **direct neighbors**, not all nodes!

# Distance-Vector: Initial State (t=0)




**At t=0, each node only knows cost to its direct neighbors:**

# Distance-Vector: Initial State (t=0)




**At t=0, each node only knows cost to its direct neighbors:**

**Node X table**

| | x | y | z |
|---|---|---|---|
| x | 0 | 2 | 7 |
| y | ∞ | ∞ | ∞ |
| z | ∞ | ∞ | ∞ |

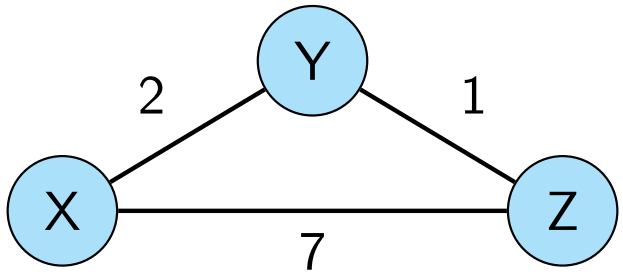
**Node Y table**

| | x | y | z |
|---|---|---|---|
| x | ∞ | ∞ | ∞ |
| y | 2 | 0 | 1 |
| z | ∞ | ∞ | ∞ |

**Node Z table**

| | x | y | z |
|---|---|---|---|
| x | ∞ | ∞ | ∞ |
| y | ∞ | ∞ | ∞ |
| z | 7 | 1 | 0 |

# Distance-Vector: Initial State (t=0)




**At t=0, each node only knows cost to its direct neighbors:**

**Node X table**

| | x | y | z |
|---|---|---|---|
| x | 0 | 2 | 7 |
| y | ∞ | ∞ | ∞ |
| z | ∞ | ∞ | ∞ |

**Node Y table**

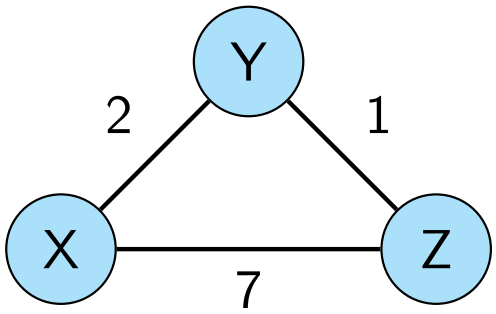
| | x | y | z |
|---|---|---|---|
| x | ∞ | ∞ | ∞ |
| y | 2 | 0 | 1 |
| z | ∞ | ∞ | ∞ |

**Node Z table**

| | x | y | z |
|---|---|---|---|
| x | ∞ | ∞ | ∞ |
| y | ∞ | ∞ | ∞ |
| z | 7 | 1 | 0 |

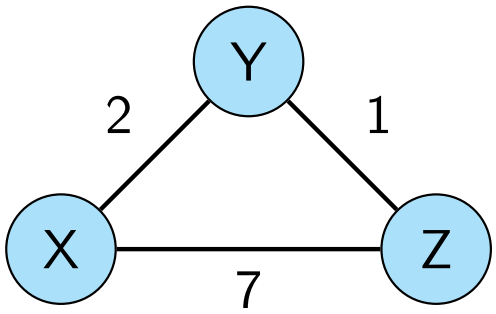
Each node sends its distance vector (highlighted row) to neighbors.

# Distance-Vector: Convergence Step-by-Step




**Step 1**: X receives $D_Y = [2, 0, 1]$ from Y

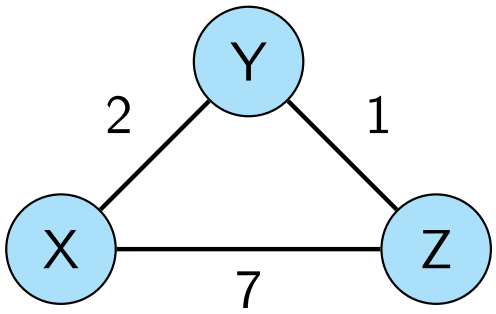
# Distance-Vector: Convergence Step-by-Step




**Step 1**: X receives $D_Y = [2, 0, 1]$ from Y

- X computes: $D_X(Z) = \min(7, 2 + 1) = 3$ via Y
- X updates its DV: $[0, 2, 3]$

# Distance-Vector: Convergence Step-by-Step



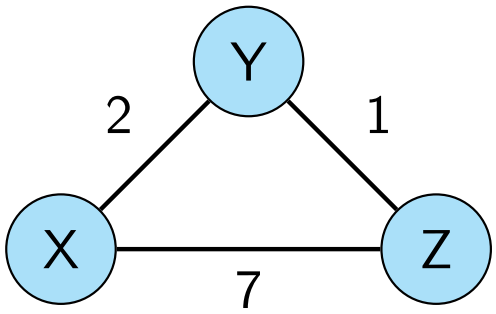

**Step 1**: X receives $D_Y = [2, 0, 1]$ from Y

- X computes: $D_X(Z) = \min(7, 2 + 1) = 3$ via Y
- X updates its DV: $[0, 2, 3]$

**Step 2**: Z receives $D_Y = [2, 0, 1]$ from Y

- Z computes: $D_Z(X) = \min(7, 1 + 2) = 3$ via Y
- Z updates its DV: $[3, 1, 0]$

# Distance-Vector: Convergence Step-by-Step




**Step 1**: X receives $D_Y = [2, 0, 1]$ from Y

- X computes: $D_X(Z) = \min(7, 2 + 1) = 3$ via Y
- X updates its DV: $[0, 2, 3]$

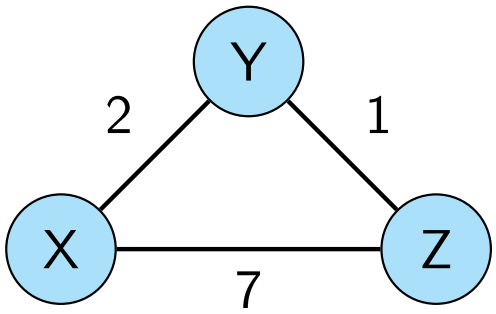
**Step 2**: Z receives $D_Y = [2, 0, 1]$ from Y

- Z computes: $D_Z(X) = \min(7, 1 + 2) = 3$ via Y
- Z updates its DV: $[3, 1, 0]$

**Step 3**: All nodes exchange updated DVs

- No changes $\Rightarrow$ **Converged!**

# Distance-Vector: Convergence Step-by-Step




**Step 1**: X receives $D_Y = [2, 0, 1]$ from Y

- X computes: $D_X(Z) = \min(7, 2 + 1) = 3$ via Y
- X updates its DV: $[0, 2, 3]$

**Step 2**: Z receives $D_Y = [2, 0, 1]$ from Y

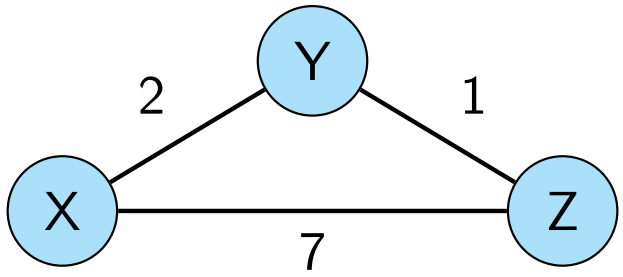
- Z computes: $D_Z(X) = \min(7, 1 + 2) = 3$ via Y
- Z updates its DV: $[3, 1, 0]$

**Step 3**: All nodes exchange updated DVs

- No changes $\Rightarrow$ **Converged!**

**Result**: X and Z learned better paths to each other (cost 3 via Y instead of direct cost 7)

# Distance-Vector: After Convergence




**After convergence, all nodes have complete routing info:**

**Node X table**

| | x | y | z |
|---|---|---|---|
| x | 0 | 2 | 3 |
| y | 2 | 0 | 1 |
| z | 3 | 1 | 0 |

**Node Y table**

| | x | y | z |
|---|---|---|---|
| x | 0 | 2 | 3 |
| y | 2 | 0 | 1 |
| z | 3 | 1 | 0 |

**Node Z table**

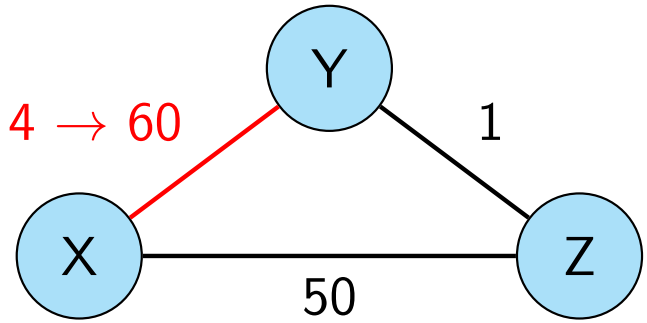
| | x | y | z |
|---|---|---|---|
| x | 0 | 2 | 3 |
| y | 2 | 0 | 1 |
| z | 3 | 1 | 0 |

X learned: $D_x(z) = \min(c(x,y) + D_y(z), c(x,z)) = \min(2+1, 7) = 3$ via Y

# Count-to-Infinity Problem

## The Problem

When a link cost **increases**, bad news travels slowly!

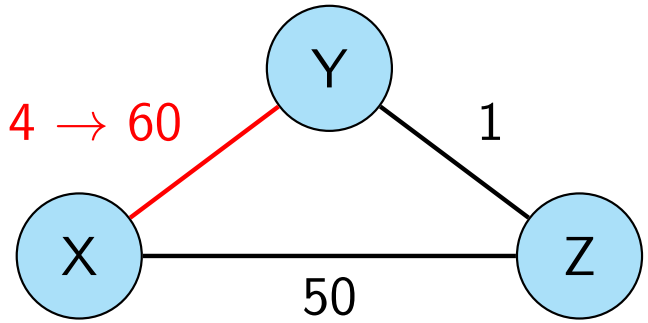



**Before**: Y routes to X directly (cost 4)
Z routes to X via Y (cost 5)

# Count-to-Infinity Problem

## The Problem

When a link cost **increases**, bad news travels slowly!




**Before**: Y routes to X directly (cost 4)
Z routes to X via Y (cost 5)

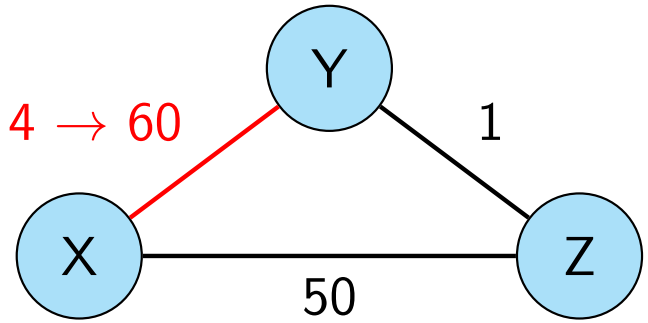
$t_0$ - **Link X-Y increases to 60:**

- Y thinks: "Z knows path to X (cost 5)"
- Y computes: $1 + 5 = 6$ via Z (wrong!)

# Count-to-Infinity Problem

## The Problem

When a link cost **increases**, bad news travels slowly!




**Before**: Y routes to X directly (cost 4)
Z routes to X via Y (cost 5)

$t_0$ - **Link X-Y increases to 60:**

- Y thinks: "Z knows path to X (cost 5)"
- Y computes: $1 + 5 = 6$ via Z (wrong!)

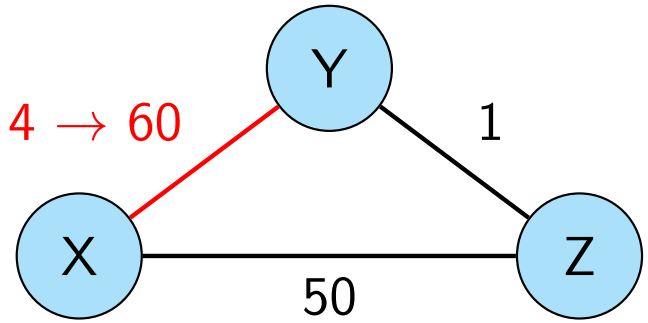
$t_1$ - **Z receives Y's update:**

- Z updates: $D_Z(X) = 1 + 6 = 7$ via Y

# Count-to-Infinity Problem

## The Problem

When a link cost **increases**, bad news travels slowly!




**Before**: Y routes to X directly (cost 4)
Z routes to X via Y (cost 5)

$t_0$ - **Link X-Y increases to 60:**

- Y thinks: "Z knows path to X (cost 5)"
- Y computes: $1 + 5 = 6$ via Z (wrong!)

$t_1$ - **Z receives Y's update:**

- Z updates: $D_Z(X) = 1 + 6 = 7$ via Y

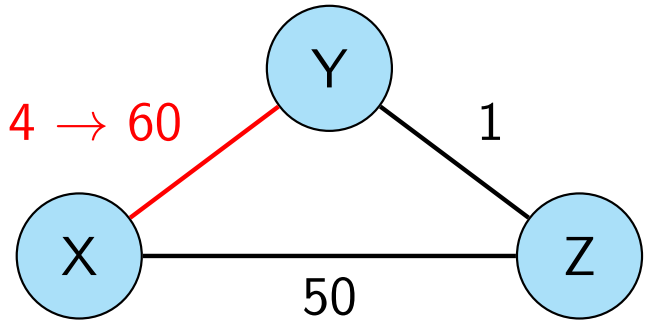
$t_2$ - **Y receives Z's update:**

- Y updates: $D_Y(X) = 1 + 7 = 8$ via Z

# Count-to-Infinity Problem

## The Problem

When a link cost **increases**, bad news travels slowly!




**Before**: Y routes to X directly (cost 4)
Z routes to X via Y (cost 5)

$t_0$ - **Link X-Y increases to 60:**

- Y thinks: "Z knows path to X (cost 5)"
- Y computes: $1 + 5 = 6$ via Z (wrong!)

$t_1$ - **Z receives Y's update:**

- Z updates: $D_Z(X) = 1 + 6 = 7$ via Y

$t_2$ - **Y receives Z's update:**

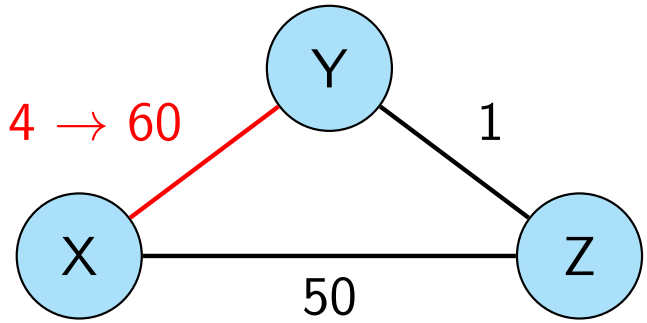
- Y updates: $D_Y(X) = 1 + 7 = 8$ via Z

**Loop continues...** until reaching 50 (direct path)

# Count-to-Infinity Problem

## The Problem

When a link cost **increases**, bad news travels slowly!




**Before**: Y routes to X directly (cost 4)
Z routes to X via Y (cost 5)

$t_0$ - **Link X-Y increases to 60:**

- Y thinks: "Z knows path to X (cost 5)"
- Y computes: $1 + 5 = 6$ via Z (wrong!)

$t_1$ - **Z receives Y's update:**

- Z updates: $D_Z(X) = 1 + 6 = 7$ via Y

$t_2$ - **Y receives Z's update:**

- Y updates: $D_Y(X) = 1 + 7 = 8$ via Z

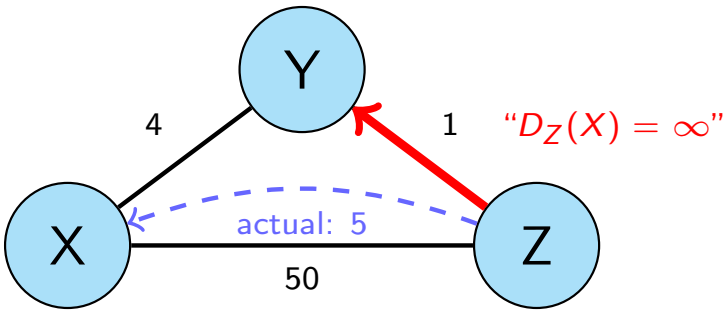
**Loop continues...** until reaching 50 (direct path)

**"Good news travels fast, bad news travels slow"** - Y and Z keep "bouncing" off each other!

# Poisoned Reverse

## Solution: Poisoned Reverse

If $Z$ routes through $Y$ to reach destination $X$, then $Z$ advertises to $Y$ that its distance to $X$ is $\infty$.




Actual: $D_Z(X) = 5$ via Y
Advertised to Y: $D_Z(X) = \infty$

**How it works:**

- Z routes to X via Y (cost 5)
- Z "lies" to Y: "my cost to X is $\infty$"
- Y won't try to route to X via Z
- Prevents 2-node routing loops!

**Limitation:**

- Does **not** solve count-to-infinity for loops involving 3+ nodes

# Link-State vs Distance-Vector

| **Property** | **Link-State (LS)** | **Distance-Vector (DV)** |
|---|---|---|
| Knowledge | Global (entire topology) | Local (neighbors only) |
| Algorithm | Dijkstra | Bellman-Ford |
| Convergence | Fast | Can be slow |
| Robustness | More robust | Count-to-infinity issue |
| Updates sent to | All nodes | Only neighbors |

**LS Advantage**

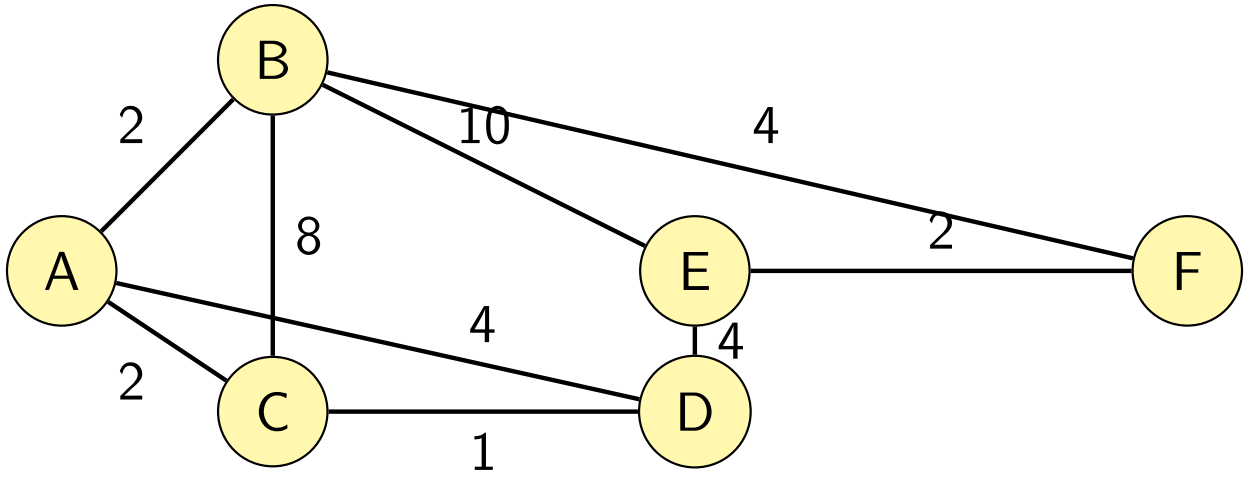
Each node can compute routes independently once it has full topology.

**DV Advantage**

Simpler, requires less memory, only local information needed.

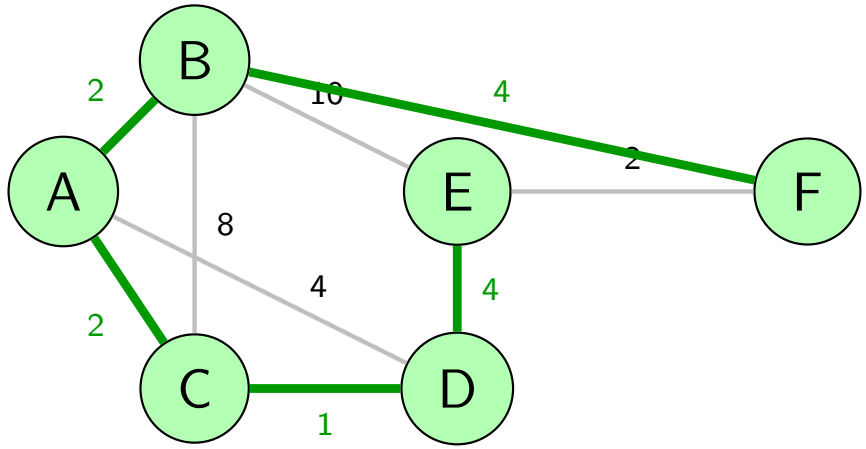
# Question: Link-State (Dijkstra)

**Fill in the Dijkstra table for node A:**



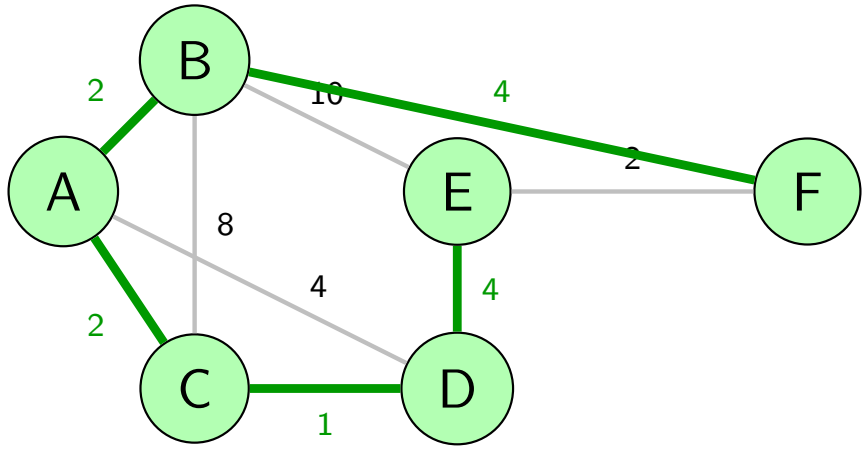

| Step | N' | D(B), p(B) | D(C), p(C) | D(D), p(D) | D(E), p(E) | D(F), p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2, A | 2, A | 4, A | $\infty$ | $\infty$ |
| 1 | ? | | | | | |
| ... | | | | | | |

# Solution: Link-State (Dijkstra)



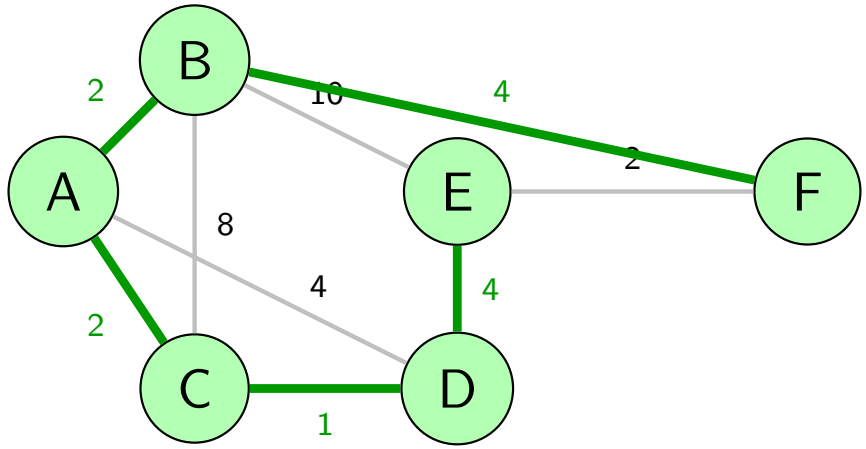

| Step | N' | D(B), p(B) | D(C), p(C) | D(D), p(D) | D(E), p(E) | D(F), p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2, A | 2, A | 4, A | $\infty$ | $\infty$ |
| | | | | | | |

# Solution: Link-State (Dijkstra)



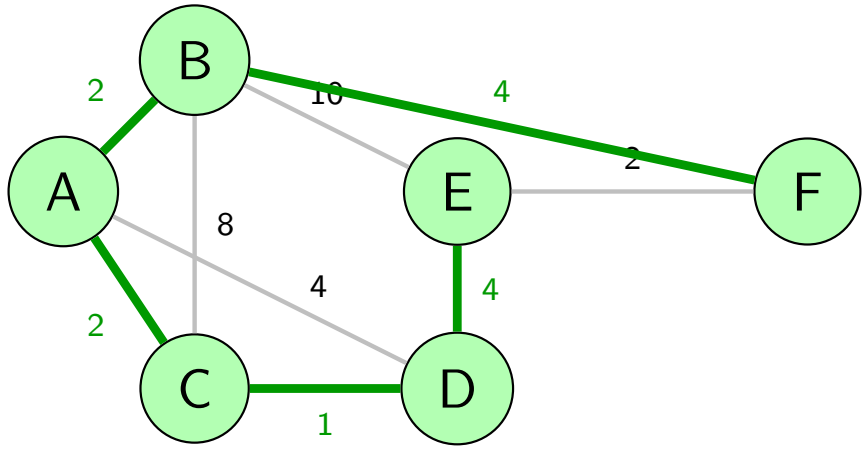

| Step | N' | D(B), p(B) | D(C), p(C) | D(D), p(D) | D(E), p(E) | D(F), p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2, A | 2, A | 4, A | $\infty$ | $\infty$ |
| 1 | AB | | 2, A | 4, A | 12, B | 6, B |

# Solution: Link-State (Dijkstra)



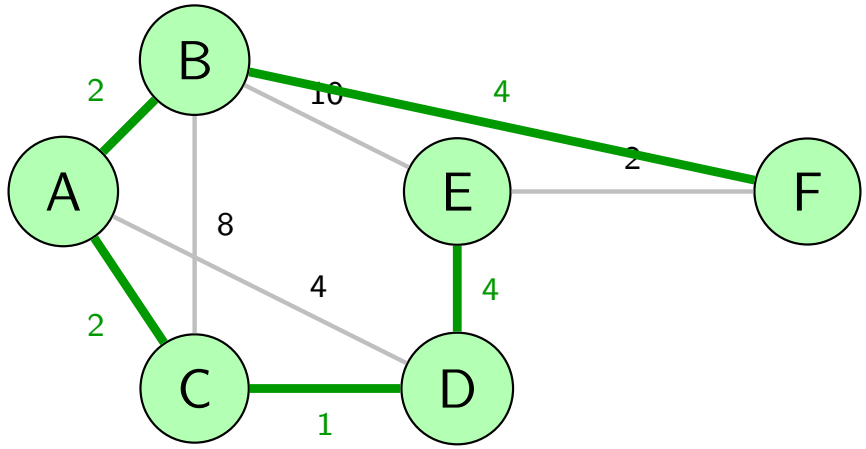

| Step | N' | D(B), p(B) | D(C), p(C) | D(D), p(D) | D(E), p(E) | D(F), p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2, A | 2, A | 4, A | $\infty$ | $\infty$ |
| 1 | AB | | 2, A | 4, A | 12, B | 6, B |
| 2 | ABC | | | 3, C | 12, B | 6, B |

# Solution: Link-State (Dijkstra)



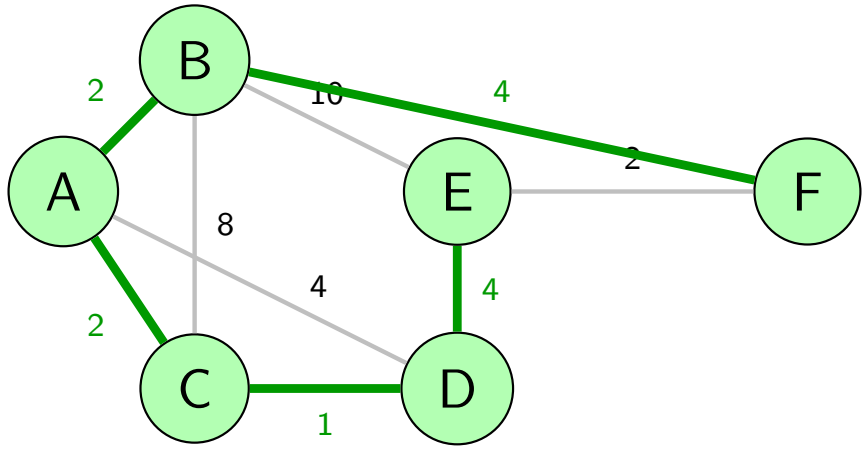

| Step | N' | D(B), p(B) | D(C), p(C) | D(D), p(D) | D(E), p(E) | D(F), p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2, A | 2, A | 4, A | $\infty$ | $\infty$ |
| 1 | AB | | 2, A | 4, A | 12, B | 6, B |
| 2 | ABC | | | 3, C | 12, B | 6, B |
| 3 | ABCD | | | | 7, D | 6, B |

# Solution: Link-State (Dijkstra)




| Step | N' | D(B), p(B) | D(C), p(C) | D(D), p(D) | D(E), p(E) | D(F), p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2, A | 2, A | 4, A | $\infty$ | $\infty$ |
| 1 | AB | | 2, A | 4, A | 12, B | 6, B |
| 2 | ABC | | | 3, C | 12, B | 6, B |
| 3 | ABCD | | | | 7, D | 6, B |
| 4 | ABCDF | | | | 7, D | |

# Solution: Link-State (Dijkstra)

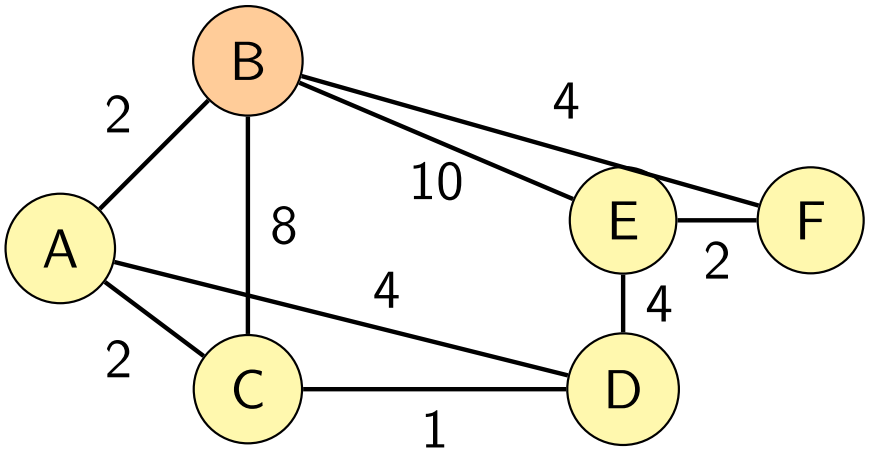



| Step | N' | D(B), p(B) | D(C), p(C) | D(D), p(D) | D(E), p(E) | D(F), p(F) |
|---|---|---|---|---|---|---|
| 0 | A | 2, A | 2, A | 4, A | $\infty$ | $\infty$ |
| 1 | AB | | 2, A | 4, A | 12, B | 6, B |
| 2 | ABC | | | 3, C | 12, B | 6, B |
| 3 | ABCD | | | | 7, D | 6, B |
| 4 | ABCDF | | | | 7, D | |
| 5 | ABCDEF | | | | | |

**Shortest paths from A**: B(2), C(2), D(3 via C), E(7 via D), F(6 via B)

# Question: Distance-Vector

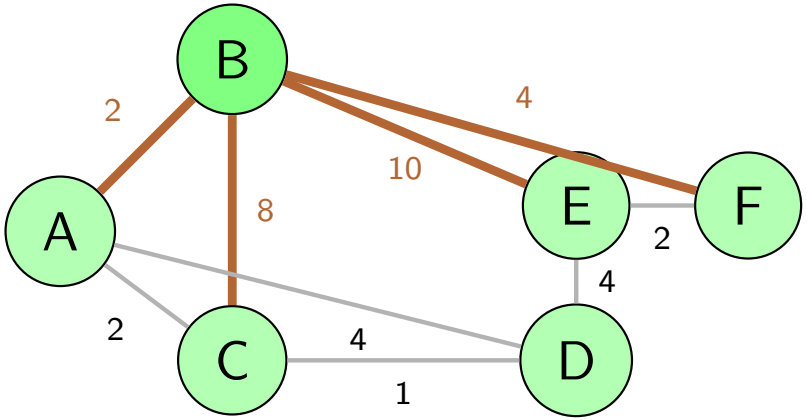
**Fill in B's distance table after DV algorithm finishes (with poisoned reverse):**



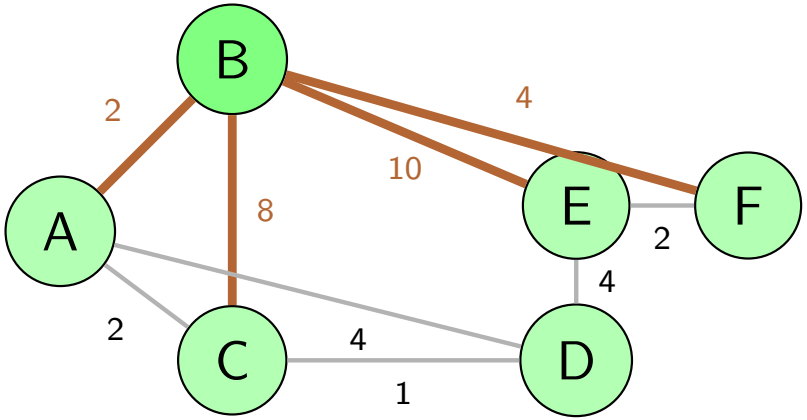

| **B's table** | **Destinations** | | | | |
|---|---|---|---|---|---|
| **Neighbors** | A | C | D | E | F |
| A | | | | | |
| C | | | | | |
| E | | | | | |
| F | | | | | |

**Hint**: Fill assuming DV converged. Account for poisoned reverse: if neighbor X routes to dest Y via B, X advertises $\infty$ for Y.

# Solution: Distance-Vector

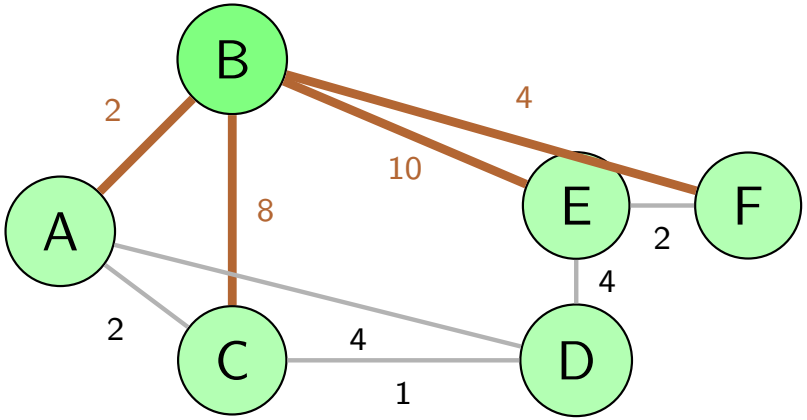


# Solution: Distance-Vector




| B's table | Destinations | | | | |
|---|---|---|---|---|---|
| **Neighbors** | A | C | D | E | F |
| A | 2 | 4 | 5 | 9 | $\infty$ |
| C | 10 | 8 | 9 | 13 | 15 |
| E | 17 | 15 | 14 | 10 | 12 |
| F | $\infty$ | 11 | 10 | 6 | 4 |

# Solution: Distance-Vector




| B's table | Destinations | | | | |
|---|---|---|---|---|---|
| **Neighbors** | A | C | D | E | F |
| A | 2 | 4 | 5 | 9 | $\infty$ |
| C | 10 | 8 | 9 | 13 | 15 |
| E | 17 | 15 | 14 | 10 | 12 |
| F | $\infty$ | 11 | 10 | 6 | 4 |

**Poisoned Reverse in action**:

- A routes to F via B $\Rightarrow$ A tells B: $D_A(F) = \infty$
- F routes to A via B $\Rightarrow$ F tells B: $D_F(A) = \infty$

# True/False: Dijkstra and MST

**Question**

The spanning tree that is built using the link-state (Dijkstra) algorithm is a minimum spanning tree.

# True/False: Dijkstra and MST

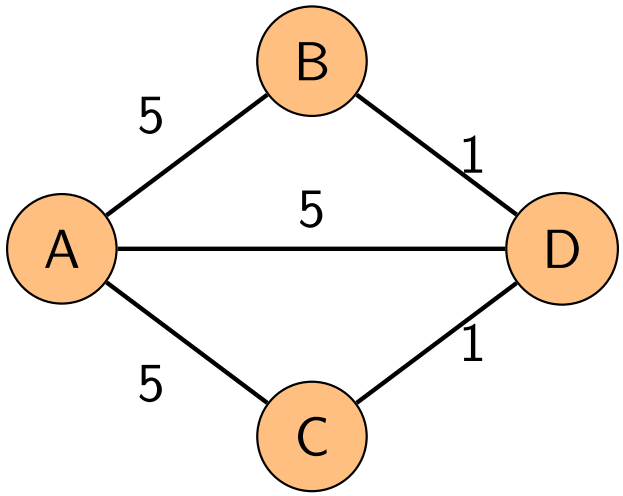
**Question**

The spanning tree that is built using the link-state (Dijkstra) algorithm is a minimum spanning tree.

**Answer: FALSE**

Dijkstra builds a **Shortest Path Tree (SPT)**, not a Minimum Spanning Tree (MST).

# True/False: Dijkstra and MST




**SPT from A**: edges A-B, A-C, A-D (total weight = 15)
**MST**: edges B-D, C-D, A-D (total weight = 7)

# True/False: DV Update Messages

**Question**

If the routing is according to "distance vector" then when there is a change in the weight of a specific edge, and the weight of the edge is **increased** compared to its previous weight, then there is **never** a need to send an update message.

# True/False: DV Update Messages

**Question**

If the routing is according to "distance vector" then when there is a change in the weight of a specific edge, and the weight of the edge is **increased** compared to its previous weight, then there is **never** a need to send an update message.

**Answer: FALSE**

When an edge weight increases, if that edge was part of the shortest path, the distance vector **must** be updated and neighbors notified.

# True/False: DV Update Messages

**Counterexample**: Consider a simple triangle A-B-C. If edge A-B increases from 2 to 6:

- B's distance to destinations via A changes
- B must recalculate and send updates to neighbors
- This is exactly the "bad news travels slow" scenario!

